Which Byte Belongs Where: A Data-Movement Characterization of a Production Visual-SLAM Workload for Memory-Centric Heterogeneous Architectures

A thesis submitted in partial fulfillment of the requirements
for the degree of

Bachelor of Engineering

by

Mayura Inomal

Department of Computer Engineering
[University Name]

July 2026

All measurements, analysis code, and result artifacts referenced in this thesis are archived in the accompanying repository (`github.com/mij001/cuvslam-stack`, MIT license); every table regenerates from committed data.

Abstract

Physical AI systems—robots, drones, and autonomous vehicles—run their perception stacks on GPUs, and their dominant cost is increasingly not arithmetic but *data movement*: the time and energy spent shuttling bytes between memory and compute. Memory-centric hardware—processing-in-memory (PiM), in/near-storage processing, and near-sensor computation—promises to attack this cost, but hardware cannot be designed for a workload that has not been measured. This thesis presents the first per-kernel, per-data-structure memory characterization of a *production* visual-SLAM system: cuVSLAM, the CUDA-accelerated simultaneous localization and mapping library deployed in NVIDIA's Isaac robotics stack.

The thesis makes three contributions. First, a *method*: GPU-DAMOV, a re-derivation of the DAMOV data-movement methodology for GPUs, in which the last-to-first miss ratio is redefined for a two-level cache hierarchy, latency-boundedness is conditioned on occupancy, and memory-divergence (coalescing) is introduced as a first-class axis with no CPU analogue. The method extends DAMOV's question—*is this code memory-bound?*—to the question an architect needs answered: *which substrate should run it, and if it stays on the GPU, which specific fault should be fixed?* Second, an *empirical* contribution: applying the method to cuVSLAM across 27 sequences from four public datasets (KITTI, EuRoC, TUM RGB-D, TUM-VI), with 0 failed runs, yields an eight-class bottleneck taxonomy (G0–G7) that is stable across inputs (91% cross-sequence class consistency), independently recovered by two unsupervised clustering algorithms (k-means and Ward hierarchical, purity ≈0.68), and 80% reproducible across two GPU microarchitectures. A novel allocation-journaling instrumentation (TaggedAllocator) joined against binary-instrumentation address traces attributes every byte of DRAM traffic to a named data structure: 274/274 allocations resolve, and 48 of 49 kernels carry a unanimous data-structure tag across all sequences. Third, a *trust* contribution: the profiled system's trajectory accuracy reproduces NVIDIA's published results across a 141-configuration matrix, and profiling itself is shown to be accuracy-neutral—on deterministic modes the profiled trajectories are bit-identical to unprofiled runs across a 192-configuration campaign.

The characterization resolves cuVSLAM's memory behaviour into a three-way split with direct architectural consequences. The image front-end is *cache-immune streaming*: its measured reuse-distance profile is flat from 64 KiB to 48 MiB, so no realizable cache helps, and the measured 1.68 MB-per-frame sensor upload argues for near-sensor consumption.

The bundle-adjustment solver is hot-persistent and largely compute-efficient: its DRAM traffic is 97% one named, pre-sized linear-system structure. The loop-closure back-end is a scattered gather (23–30 sectors per warp access) over a keyframe database whose GPU-resident state is a fixed 6.7 MB buffer while the session-scale store grows host-side—the in-storage-processing case—and 92.6% of the scan kernel's DRAM volume is register-spill traffic, a compiler artifact invisible to counter-only studies. In total, 60–72% of GPU time carries PiM affinity under a sensitivity-tested classification. A measured whole-run energy baseline (34.7 J) and host-side I/O characterization (66 MB storage ingestion; 708 MB peak host memory) complete the evidence base for the follow-on architecture study, for which simulator-configuration groundwork is laid.

Beyond its findings, the thesis documents two self-corrections in which independent measurement methods initially disagreed and were reconciled—one reversing a published-in-project claim—and argues that this disagree-then-agree arc, with all artifacts on record, is precisely what distinguishes measurement science from advocacy.

Contents

List of Tables

List of Abbreviations

AI	Arithmetic Intensity (FLOPs per DRAM byte)
APE / ATE	Absolute Pose / Trajectory Error
ARI	Adjusted Rand Index
BA	Bundle Adjustment
CDF	Cumulative Distribution Function
CoV	Coefficient of Variation (std. dev. ÷ mean)
CUDA	Compute Unified Device Architecture (NVIDIA GPU programming model)
DAMOV	the Data-Movement bottleneck methodology of Oliveira et al. [1]
DRAM	Dynamic Random-Access Memory (GPU main/“global” memory)
FLOP	Floating-Point Operation
FMA	Fused Multiply–Add
GFTT	Good Features To Track (Shi–Tomasi corner detector)
GPU	Graphics Processing Unit
H2D / D2H	Host-to-Device / Device-to-Host (PCIe copies)
IMU	Inertial Measurement Unit
ISP	In-Storage Processing (computational storage); in camera contexts, Image Signal Processor
LFMR	Last-to-First Miss Ratio
LK	Lucas–Kanade optical flow
LMDB	Lightning Memory-mapped Database
MPKI	(DRAM sector) Misses Per Kilo-Instruction
NDP	Near-Data Processing
ncu	NVIDIA Nsight Compute (hardware-counter profiler)
nsys	NVIDIA Nsight Systems (timeline profiler)
NVBit	NVIDIA Binary Instrumentation Tool
NVTX	NVIDIA Tools Extension (code-range annotations)
PCIe	Peripheral Component Interconnect Express (CPU–GPU link)
PiM / PnM	Processing-in-Memory / Processing-near-Memory
QoR	Quality of Results (instrumented-vs-baseline output equivalence)

RMSE	Root-Mean-Square Error
RPE / RTE	Relative Pose / Trajectory Error
SLAM	Simultaneous Localization And Mapping
SM	Streaming Multiprocessor
SoL	Speed-of-Light (utilization as % of theoretical peak)
SRAM	Static Random-Access Memory (on-chip storage)
VIO	Visual-Inertial Odometry
VRAM	Video RAM (the GPU's DRAM)

Chapter 1

Introduction

1.1 Motivation: Physical AI meets the memory wall

A robot that moves through the world must answer two questions continuously and in real time: *where am I?* and *what does my surrounding look like?* The computational discipline that answers both at once from camera images is visual Simultaneous Localization And Mapping (SLAM), and in production robotics it runs on a GPU. The workload studied in this thesis is cuVSLAM [2], NVIDIA's CUDA-accelerated visual SLAM library, shipped inside the Isaac robotics stack and deployed on real robots. It is, by construction, a representative workload of what is now called *Physical AI*: machines that sense and act in the physical world under hard latency and energy budgets.

For such machines, the binding constraint on the processor is no longer arithmetic. Four decades of technology scaling have made floating-point operations nearly free relative to the cost of *moving the operands*: fetching a word from off-chip DRAM costs orders of magnitude more energy than computing with it, and the gap widens every generation. This is the *memory wall* [3], the modern face of the von Neumann bottleneck. For a battery-powered robot the consequence is direct: joules spent moving pixels and map entries are joules not spent on flight time or actuation.

The architecture community's response is *memory-centric* hardware: processing-in-memory (PiM), which places compute engines at or inside the DRAM devices [4, 5]; in-storage or near-storage processing, which executes queries where the data is stored [6, 7]; and near-sensor processing, which consumes pixel streams before they ever reach main memory. Commercial silicon exists for each of these ideas [8, 9, 10]. What does *not* exist is a principled way to decide, for a given production workload, *which* of its computations belongs on *which* substrate. That decision requires measurement: a *characterization* of exactly how the workload moves data.

1.2 Problem statement

The problem this thesis addresses can be stated in one sentence:

> *For each GPU kernel of a production visual-SLAM system, determine—by measurement, at the granularity of named data structures, with quantified confidence—whether its bottleneck is data movement, which class of data movement, and consequently which hardware substrate (GPU, CPU, DRAM-PiM, scatter-capable PiM, near-sensor, or in/near-storage) is the appropriate home for it.*

Three properties make this problem hard, and each drives a contribution of this thesis:

1. **The reference methodology is CPU-shaped.** The established data-movement characterization methodology, DAMOV [1], was built for CPUs. GPUs violate its assumptions in specific ways: they have a two-level (not three-level) cache hierarchy; they hide memory latency with massive thread parallelism rather than stalling; and they possess a failure mode with no CPU analogue—*memory divergence*, where the 32 threads of a warp scatter their accesses and waste up to 8× the fetched bandwidth. A GPU characterization must adapt the methodology, not merely reuse it (Chapter 4).

2. **Kernel-level claims are not architect-level claims.** A profiler can report that a kernel is memory-bound; an architect needs to know *which data structure* the traffic belongs to—an image, a solver matrix, or a growing keyframe database—because the substrate verdict differs for each. Attributing every byte of DRAM traffic to a named allocation in a closed-source production binary requires purpose-built instrumentation (Chapter 8).

3. **Measurements must be proven trustworthy.** A hostile reviewer will ask, in order: Was the workload even computing correctly? Did the instrumentation change what was measured? Are the numbers stable run to run? Would the classification survive different thresholds, different clustering algorithms, different hardware? Each of these questions is answered with a dedicated experiment in this thesis (Chapters 5 and 6).

1.3 Research questions

RQ1 Can DAMOV's data-movement methodology be re-derived for GPUs such that each of its components—screening, machine-independent locality analysis, bottleneck classification, and robustness validation—has a measured GPU analogue? (Chapter 4; validated in Chapters 6–7.)

RQ2 What are the memory-bottleneck classes of a production visual-SLAM workload, and are they properties of the kernels themselves or artifacts of particular inputs, thresholds, or machines? (Chapter 6.)

RQ3 Which named data structures receive each kernel's DRAM traffic, and what fraction of that traffic is program data versus compiler-generated scratch? (Chapter 8.)

RQ4 What is the resulting substrate placement for the workload's computation, what fraction of GPU time is offload-eligible, and what measured baselines (energy, host I/O) must a follow-on architecture study improve upon? (Chapter 9.)

RQ5 Can the entire measurement apparatus be shown to be accuracy-neutral—i.e., that the characterized system is a correctly functioning SLAM whose outputs are unchanged by observation? (Chapter 5.)

1.4 Contributions

1. **GPU-DAMOV, a methodology** (Chapter 4). A component-by-component re-derivation of DAMOV for GPUs: LFMR redefined for a two-level hierarchy; latency-boundedness conditioned on occupancy; a coalescing axis added; DAMOV's simulated intervention experiment replaced by two *measured* interventions (a clock-domain sensitivity sweep and a direct reuse-distance cache-capacity curve) with the simulated third axis prepared as generated configuration. The method ends one step later than DAMOV: not at "memory-bound" but at a per-kernel *substrate verdict* plus, when the verdict is "stay on GPU," the named architectural fault to fix.

2. **The first per-data-structure memory characterization of a production SLAM system** (Chapters 6–9). 49 kernels × 27 sequences × 4 datasets × 192 configuration variants, with an eight-class taxonomy (G0–G7) in which class membership is 91% consistent across sequences, independently recovered by two clustering algorithms, and 80% reproducible across two GPU microarchitectures.

3. **TaggedAllocator attribution** (Chapter 8). A three-layer instrumentation—source-level allocation journaling with call stacks, driver-level allocation lifetime capture, and a streaming join against binary-instrumentation address traces—that resolves 274/274 allocations and gives 48/49 kernels a unanimous data-structure tag. It surfaces a finding invisible to counters: 92.6% of the loop-closure scan's DRAM volume is register-spill scratch, not data.

4. **A validated trust chain** (Chapter 5). A 141-configuration accuracy matrix reproducing the vendor's published trajectory accuracy; a 192-configuration campaign showing

profiling is accuracy-neutral (bit-identical trajectories on deterministic modes); a measured noise floor (run-to-run CoV 0.14% at locked clocks); and a ±25% threshold sensitivity analysis.

5. **A reusable, workload-agnostic toolchain** (released under MIT license): a single-file experiment runner, a config-mutation regime, a three-profiler harness with adapters for arbitrary GPU workloads, a stdlib-only analysis library that regenerates every table from committed CSVs without a GPU, and an interactive evidence dashboard.

6. **Two documented self-corrections** in which independent methods disagreed and were reconciled on record—one reversing an earlier project-internal conclusion (Chapter 7)—presented as a feature of the methodology: the pipeline catches its own errors.

1.5 Scope and non-goals

This thesis is a *characterization*, deliberately stopping at measured candidacy plus an analytical placement bound. It does *not* simulate a PiM or in-storage design and therefore reports no end-to-end speedup or energy delta for the proposed substrates; that is the follow-on architecture study, for which this thesis generates the simulator configurations and measures the baselines (Section 11.3). It characterizes one workload family—cuVSLAM—arguing representativeness from production deployment rather than breadth across SLAM implementations; the toolchain's adapter mechanism is the demonstrated path to breadth. Hardware evaluation uses one desktop-class GPU as the primary instrument, with a second microarchitecture used for cross-device validation of the classification.

1.6 Thesis organization

Chapter 2 builds the technical background from first principles: the GPU execution model, the memory hierarchy and its five address spaces, SLAM and cuVSLAM, the candidate substrates, and the measurement instruments. Chapter 3 situates the work among data-movement characterizations, PiM systems, and SLAM acceleration studies. Chapter 4 presents GPU-DAMOV: metrics, decision tree, taxonomy, placement model, and the experimental setup. Chapter 5 establishes trust: accuracy validation, profiler neutrality, and measurement rigor. Chapter 6 reports the kernel-level characterization and its four-way validation. Chapter 7 reports the machine-independent locality analysis and the first self-correction. Chapter 8 reports the data-structure attribution and the register-spill finding. Chapter 9 synthesizes the substrate verdicts, the placement model, and the measured energy and host-I/O baselines. Chapter 10 enumerates threats to validity. Chapter 11 concludes with

the phased roadmap. Appendices provide a glossary, the artifact/reproducibility description, and extended result tables.

Chapter 2

Background

This chapter builds every concept the thesis depends on, in dependency order: the GPU execution model (§2.1), the memory hierarchy and its address spaces (§2.2), why data movement dominates cost (§2.3), visual SLAM and cuVSLAM (§2.4), the candidate hardware substrates (§2.5), and the measurement instruments (§2.6).

2.1 The GPU execution model

A CPU optimizes the latency of one instruction stream; a GPU optimizes the throughput of tens of thousands of them. NVIDIA GPUs are programmed through CUDA: the programmer writes a *kernel*—one function—and launches it over a grid of thousands to millions of *threads*, each processing a different data element. Threads are grouped in fixed bundles of 32 called *warps*; the 32 *lanes* of a warp execute the same instruction in lockstep on different data. Warps are grouped into thread blocks, and blocks are scheduled onto *streaming multiprocessors* (SMs)—the GPU's core unit. The primary instrument of this thesis, an NVIDIA RTX 2000 Ada generation GPU (compute capability `sm_89`), has 22 SMs, a 25 MB shared L2 cache, and 16 GB of GDDR6 DRAM.

Two warp-level behaviours matter throughout this thesis:

Divergence. If a branch disables some lanes of a warp, the disabled lanes do no useful work. The counter `mean_active_lanes` measures this; 32.0 means fully converged. Every cuVSLAM kernel measured in this thesis runs at 32.0 active lanes—divergence is *not* cuVSLAM's problem, which immediately focuses attention on memory behaviour.

Coalescing. When a warp issues a load, the hardware merges the 32 lane addresses into fetches of 32-byte *sectors* (four sectors form a 128-byte cache line). If the lanes address 32 consecutive words, a few sectors satisfy the whole warp—a *coalesced* access. If the lanes scatter, the hardware may fetch up to 32 sectors and discard most of each one—a *scattered gather* that wastes up to 8× the useful bandwidth. The metric *sectors per request* quantifies this ($\approx$4 = perfectly coalesced for 4-byte accesses; $\rightarrow$32 = fully scattered). Coalescing has

no CPU analogue and is one of the three reasons DAMOV cannot be applied to GPUs unchanged (Chapter 4).

Occupancy is the fraction of an SM's maximum concurrent warps that are resident. It is the GPU's latency-hiding mechanism: while one warp waits for memory, the scheduler issues another. High occupancy can hide DRAM *latency* entirely; nothing hides a *bandwidth* shortfall. This asymmetry—latency hidable, bandwidth not—shapes the entire bottleneck taxonomy of Chapter 4.

2.2 The memory hierarchy and the five address spaces

GPU storage forms the usual pyramid: per-thread *registers* (fastest, smallest); per-SM on-chip storage of 100 KB serving as L1 cache and programmer-managed *shared memory*; the chip-wide L2 cache (25 MB); the GPU's DRAM (16 GB, reachable at a measured 205 GB/s on the locked instrument, roughly 100× the latency of a register); host RAM across the PCIe bus; and persistent storage. Each level downward costs roughly an order of magnitude more energy per byte.

Independent of this physical pyramid, every memory *instruction* addresses one of several logical *spaces*, and confusing them invalidates analyses (as Chapter 7 demonstrates on this very project):

Global the program's real data in DRAM—everything allocated with `cudaMalloc`. Instructions: `LDG`/`STG`. All DRAM-traffic and locality claims in this thesis are about global space.

Shared the on-chip per-SM scratchpad (`LDS`/`STS`). Used for convolution tiles, sort staging, reduction trees. *Not DRAM*: traffic here says nothing about the memory wall.

Local a per-thread region that is physically *in DRAM* but logically compiler scratch: when a kernel needs more temporaries than its register budget, the compiler *spills* them here (`LDL`/`STL`). Local addresses are interleaved across threads, so spill traffic is *perfectly coalesced by construction*—a fact that produced this project's most instructive measurement artifact (§7.3).

Constant, texture small read-only parameter storage, and image reads through dedicated texture units. Texture fetches do not appear in the address-trace instrument used here; the implications are bounded in §8.6.

2.3 The memory wall

Since the mid-1990s, processor arithmetic throughput has grown far faster than memory bandwidth or latency has improved—the *memory wall* [3]. On contemporary hardware, a

double-precision multiply costs picojoules while a DRAM fetch of its operands costs hundreds of picojoules to nanojoules; crossing PCIe to host memory or storage costs more still. For low-arithmetic-intensity codes—image pipelines are the canonical case—the processor mostly waits. The roofline model [11] makes this quantitative (§4.3.4): performance is bounded by min(peak compute, AI × peak bandwidth), where arithmetic intensity AI is FLOPs per DRAM byte. A kernel under the sloped part of that roof cannot be helped by more arithmetic units—only by moving fewer bytes, or moving them a shorter distance. The latter option is the premise of memory-centric architecture.

2.4 Visual SLAM and cuVSLAM

Visual SLAM estimates a camera's trajectory while building a map of the environment [12, 13]. The canonical pipeline has four stages:

1. **Front-end.** Each incoming frame is converted to grayscale; an *image pyramid* (the frame at successively halved resolutions) and gradient images are built; distinctive corners are detected (Good Features To Track, the Shi–Tomasi criterion) and tracked frame-to-frame with Lucas–Kanade optical flow. This stage runs on every frame at camera rate.

2. **Visual odometry.** From tracked features, the camera's motion is estimated by solving a nonlinear least-squares problem.

3. **Bundle adjustment (BA).** Poses and 3-D landmark positions over a window are refined jointly. Numerically, each iteration builds and solves a linear system (a Hessian reduced by the Schur complement to eliminate landmark variables); this thesis names that structure `ba_linear_system` and measures precisely which kernels stream it.

4. **SLAM layer.** Selected frames become *keyframes*; their feature descriptors enter a *keyframe database*. When the camera revisits a mapped place, a database scan detects the *loop closure* and the accumulated drift of the whole trajectory is corrected. Loop closure is rare (it fires on revisits) but is what makes long-session maps consistent —and its database is the only data structure that *grows* with session length, a fact of central architectural consequence (§8.3).

cuVSLAM [2] is NVIDIA's production implementation: GPU-resident front-end and solvers, a host-side keyframe/landmark store backed by LMDB (a memory-mapped embedded database), and support for stereo, RGB-D (color plus depth), monocular, and inertial (IMU-fused) camera configurations, in odometry-only or full-SLAM modes, with synchronous and asynchronous SLAM variants. This thesis treats cuVSLAM strictly as

a measured artifact: its algorithms are never modified; instrumentation is additive and switchable, and Chapter 5 proves it output-neutral.

Accuracy of a SLAM system is scored against *ground truth* (the true trajectory from motion capture, GPS/INS, or an external tracker) using two standard metrics [14, 15]: *absolute pose/trajectory error* (APE/ATE)—the RMS position difference after optimal alignment (SE(3) for metric-scale systems; Sim(3) when monocular scale is free [16]), and *relative pose/trajectory error* (RPE/RTE)—drift over fixed-length segments, expressed as a percentage, which is scale-independent and therefore comparable across kilometre-scale (KITTI) and room-scale (EuRoC, TUM) trajectories.

2.5 Candidate substrates for memory-centric execution

The thesis evaluates each kernel's fit against five homes:

GPU (status quo). Best when there is real arithmetic per byte or reuse the caches can capture.

CPU/host. Best for tiny, launch-overhead-dominated kernels whose GPU occupancy is negligible.

DRAM-PiM (near-bank). Compute placed at DRAM banks exposes internal bandwidth several times the external interface and removes the interconnect-energy toll per byte. Commercially realized in UPMEM's DPUs [8], Samsung's HBM-PIM (function-in-memory DRAM) [9], and SK hynix's GDDR6-AiM [10]. The natural tenant is *streaming, bandwidth-bound* work: much data, little arithmetic, no reuse.

Scatter-capable PiM. Irregular gather/scatter defeats both caches and coalescing; near-memory engines that issue fine-grained random accesses (in the lineage of Tesseract and PIM-enabled instructions [17, 18]) are the architectural answer when the access pattern cannot be fixed in software.

Near-sensor SRAM / ISP. Per-frame pixel processing can execute adjacent to the image sensor, consuming the stream before it is ever written to DRAM. The measured 1.68 MB-per-frame host-to-device upload (§6.7) is exactly the traffic such placement removes.

In/near-storage processing. When a growing database is scanned episodically, the scan can execute inside the storage device [6, 7], returning matches rather than streaming the store through host memory, PCIe, and DRAM.

2.6 The measurement instruments

No single tool sees everything; the methodology composes four views.

Nsight Systems (nsys) records the timeline: every kernel launch, its duration, every PCIe copy, with near-zero overhead. It provides the screening signal (which kernels matter)

and, through NVTX ranges, the kernel-to-pipeline-stage mapping.

Nsight Compute (ncu) reads hardware performance counters per kernel by *replaying* each kernel multiple times. It is precise about *how much* (bytes, hit rates, stall cycles) but blind to *which addresses*. Replay makes it expensive, so captures are bounded to windows and to a curated metric set of ∼15–30 counters; requesting the full set is infeasible on small-memory GPUs.

NVBit [19] rewrites GPU machine code at load time to invoke a callback on every memory instruction, yielding the exact per-warp address stream—ground truth beneath ncu's aggregates—at 100–1000× slowdown. The tool used here (`mem_trace`, extended with launch-window and kernel-name gating, and with an allocation-event sidecar) makes gigabyte-scale traces feasible and, joined with the allocation journal of Chapter 8, attributable.

NVTX + TaggedAllocator name things. cuVSLAM ships with NVTX stage annotations that are compiled out by default at two independent gates; this project's instrumentation patch enables them, making the kernel-to-stage table a *measured* artifact rather than a guess from kernel names. The TaggedAllocator (Chapter 8) journals every GPU allocation with its call stack, giving each buffer a durable data-structure name.

A standing epistemic ladder orders these instruments: *counters say how much; traces say which addresses; the allocation journal says which data structure; NVTX says which pipeline stage.* Every claim in this thesis cites the highest rung available, and the two places where rungs disagreed became findings (§7.3, §**??**).

Chapter 3

Related Work

3.1 Data-movement characterization methodologies

The direct ancestor of this thesis is DAMOV [1], which systematized the question "which functions of a workload are data-movement-bound, and would near-data processing help?" for CPUs. Its three-step recipe—screen the functions that matter; characterize them with hardware counters plus an architecture-independent locality analysis; classify them into bottleneck families *derived from the data by clustering*—is adopted here wholesale, and its central metric, the last-to-first miss ratio (LFMR: of the requests that miss the first cache, the fraction that also miss the last and reach DRAM), is retained in redefined form. DAMOV's validation discipline is also adopted: frozen thresholds stress-tested on held-out inputs, an independent clustering algorithm reproducing the classes, and cache-size sweeps confirming class behaviour. Chapter 4 enumerates, component by component, the GPU analogue of each DAMOV element and the three places where the GPU forced a redesign rather than a translation. Where DAMOV's third step is a simulator-based intervention experiment (host vs. host-with-prefetcher vs. NDP across a core-count sweep in ZSim/Ramulator), this thesis substitutes two *measured* interventions on silicon—independent core- and memory-clock scaling, and the reuse-distance cache-capacity curve—and defers the simulated axis to generated configuration (§4.7).

On the GPU side, the counter-metric curation used here follows the NCU-based roofline methodology of Cao et al. [20], who demonstrated a disciplined path from Nsight Compute counters to roofline placement for GPU database kernels; this thesis reuses their metric taxonomy (extended with stall-reason and occupancy axes) and their practice of measuring, rather than quoting, the roofline ceilings. The roofline model itself is due to Williams et al. [11]. Reuse-distance (stack-distance) analysis originates with Mattson et al. [21]; its use here as a *measured* hit-rate-versus-capacity curve from binary-instrumentation traces (§**??**) is the GPU translation of DAMOV's architecture-independent locality step, with one addition the CPU literature does not need: filtering by memory space, without which register-spill

and scratchpad traffic corrupt the locality signal (§7.3).

3.2 Processing-in-memory and near-data systems

The PiM literature spans three decades; the modern wave is surveyed by Mutlu et al. [4] and Ghose et al. [5]. Architectures with direct bearing on this thesis's verdicts include Tesseract [17] (near-memory graph processing—the lineage for scatter-capable PiM), PIM-enabled instructions [18] (fine-grained offload of cache-defeating operations), TOM [22] (transparent offload of GPU code blocks to 3D-stacked memory, selected by a compiler bandwidth-benefit analysis— methodologically the closest GPU-offload ancestor), and the workload-driven placement study of Boroumand et al. [23] for consumer workloads. Commercial near-bank silicon—UPMEM DPUs [8], Samsung HBM-PIM [9], SK hynix GDDR6-AiM [10]—establishes that the substrates this thesis assigns work to are realizable, not hypothetical. In-storage processing is grounded in Smart-SSD query offload [6] and Biscuit [7]. What none of these provide is the *workload side* for Physical AI: a measured statement of which SLAM computations and which SLAM data structures want which substrate. That is the gap this thesis fills.

3.3 SLAM systems and their evaluation

The SLAM algorithmic literature is mature; Cadena et al. [12] survey it. The systems measured against in this thesis's accuracy validation are cuVSLAM itself, as documented in NVIDIA's technical report [2], with ORB-SLAM2/3 as the standard open-source references [24, 25]. Benchmarking SLAM *as a computational workload* (rather than for accuracy) was pioneered by SLAMBench [26], which profiled dense SLAM across CPU/GPU platforms at whole-pipeline granularity. This thesis differs in three ways: it measures a *production* sparse V-SLAM stack rather than a research kernel; it descends to per-kernel and per-data-structure granularity; and its output is a substrate placement, not a platform comparison. Trajectory-accuracy methodology follows the TUM benchmark conventions [14] and the evaluation tutorial of Zhang and Scaramuzza [15], with Umeyama alignment [16]. The datasets used—KITTI [27], EuRoC [28], TUM RGB-D [14], TUM-VI [29], and ICL-NUIM [30]—are the community's standard ground-truth corpora.

3.4 GPU instrumentation

Binary instrumentation of GPU machine code is provided by NVBit [19], on which this thesis's address tracing is built (with two extensions: launch-window/kernel-name gating to bound trace volume, and an allocation-event sidecar that timestamps allocator activity in

the trace's own launch-id clock). GPU simulation and power modelling—Accel-Sim [31] and AccelWattch [32]—appear in this thesis only as the target of generated configuration for the follow-on study (§11.3); consistent with a standing project rule, no simulated number is reported in the results chapters.

3.5 Positioning

In one sentence per axis: relative to DAMOV, this thesis is the GPU re-derivation plus the extension from bottleneck class to substrate verdict; relative to GPU profiling practice, it adds machine-independent locality and named-data-structure attribution on top of counters; relative to the PiM systems literature, it supplies the missing measured workload analysis for a deployed Physical-AI perception stack; relative to SLAM benchmarking, it is the first study at the granularity where architectural decisions are actually made—the kernel and the allocation.

Chapter 4

GPU-DAMOV: Methodology and Experimental Setup

This chapter defines the method. Section 4.1 gives the pipeline; §4.2 the instruments and workloads; §4.3 every metric with its formula; §4.4 the classifier and taxonomy; §4.5 the substrate-verdict and fault mapping; §4.6 the placement model; and §4.7 the component-by-component parity argument that GPU-DAMOV is complete at DAMOV's granularity (RQ1).

4.1 The six-step pipeline

`Capture` → `Screen` → `Classify` → `Attribute` → `Validate` → `Verdict`

Capture runs the workload under the three profilers at locked clocks: nsys for the timeline, ncu for counters (bounded launch windows, curated metric set), NVBit for address traces (windowed). Every run also records whole-run energy (GPU power sampled and integrated), host-side I/O (storage bytes read, memory-mapped page-ins, peak host RSS), and—always— trajectory accuracy against ground truth, so that no measurement of a malfunctioning run can enter the evidence. **Screen** ranks kernels by share of GPU time from the timeline; kernels below the attention floor are screened out, not classified (they receive class G0). **Classify** feeds each surviving kernel's metrics to the ordered decision tree (§4.4). **Attribute** joins address traces against the allocation journal to name each kernel's traffic (Chapter 8). **Validate** stress-tests everything: threshold sensitivity, unsupervised clustering, cross-device agreement, a known-truth calibration suite, and profiler-neutrality checks (Chapters 5–6). **Verdict** maps class × data-structure persistence × features to a substrate, with a named on-GPU fault when the verdict is "stay."

4.2 Experimental setup

4.2.1 Hardware

The primary instrument is an NVIDIA RTX 2000 Ada generation GPU (`sm_89`): 22 SMs, 100 KB unified L1/shared per SM, 25 MB L2, 16 GB GDDR6. A second, deliberately different microarchitecture—a GeForce MX450 laptop part (`sm_75`, 2 GB, 25 W)—serves only for cross-device validation of the classification (§6.4). The host workstation runs driver 575.64.05 with CUDA 12.9.1; profilers are Nsight Compute 2025.2.1.3 and Nsight Systems 2025.3.2; address tracing uses NVBit 1.8 [19]. This exact combination matters operationally (binary instrumentation constrains the driver generation, which in turn constrains profiler versions) and is documented in the artifact so the environment is reconstructible (Appendix B).

4.2.2 Locked clocks and measured ceilings

GPUs modulate frequency continuously for power and thermals; two runs at different clocks are not comparable. All measurements lock the graphics clock at 1620 MHz and the memory clock at 7001 MHz. The effect is not cosmetic: the median run-to-run coefficient of variation (CoV = standard deviation ÷ mean over repeated runs) of kernel time drops from **49.6%** unlocked on a thermally limited laptop, and 9.3% warmed, to **0.14%** locked on the workstation. At 0.14%, run-to-run noise is three orders of magnitude below every effect size reported in this thesis.

Roofline ceilings are *measured*, not quoted from datasheets: at the locked clocks, sustained DRAM bandwidth is 205.0 ± 0.1 GB/s and FP32 throughput is 5445 ± 3 GFLOP/s (microbenchmarks in the artifact). All roofline placements and DRAM speed-of-light percentages in this thesis are relative to these measured ceilings.

4.2.3 Workloads and datasets

The characterization corpus is 27 sequences across four public ground-truth datasets: KITTI odometry 00–10 (automotive stereo, kilometre scale) [27]; EuRoC MAV MH01–05, V101–103, V201–203 (drone stereo + IMU, room/hall scale) [28]; TUM RGB-D fr3 sequences including the loop-closure-dense `long_office_household` (handheld RGB-D) [14]; and TUM-VI (visual-inertial) [29]. The accuracy-validation matrix (Chapter 5) additionally includes ICL-NUIM [30]. Sequences span indoor and outdoor, room-scale and street-scale, loop-closing and deliberately loop-free trajectories (KITTI 01 and 04 close no loops and serve as a control). Pipeline modes exercised: odometry-only and full SLAM; synchronous and asynchronous SLAM; GPU and CPU SLAM back-ends; stereo, RGB-D, monocular, and inertial sensor configurations. In total the configuration regime materializes 141 accuracy configurations

and 192 profiling-coverage variants from a small set of human-owned base configurations via a deterministic mutation pass, so the whole matrix is reproducible byte-for-byte.

4.2.4 Capture discipline

Three rules are enforced throughout. (1) *Traces are never used for timing*: instrumented runs are 5–1000× slower by design; all timing comes from uninstrumented locked-clock runs. (2) *Windows must be audited*: windowed captures risk missing sparsely-launching kernels, so a coverage audit diffs every sequence's full launch map against the captured set and iteratively gap-fills until zero kernels are missing (§8.2). (3) *Extensive before intensive*: time, bytes, and instruction counts are summed across a kernel's launches first and ratios are formed once; averaging per-launch ratios would weight a microsecond launch equally with a millisecond one.

4.3 Metrics

Each metric reduces hardware counters or trace records to one interpretable feature per kernel.

4.3.1 Speed-of-light utilizations

Nsight Compute reports each subsystem's utilization as a percentage of its theoretical peak: *memory SoL* (the memory pipeline), *compute SoL* (the arithmetic pipelines), and *DRAM SoL* (DRAM bandwidth). DRAM SoL $\geq$ 50% is read as bandwidth saturation; compute SoL substantially above memory SoL indicates compute-boundedness.

4.3.2 LFMR (GPU form)

$$\text{LFMR} \;=\; 1 - (\text{L2 hit rate}). \tag{4.1}$$

DAMOV's CPU definition—misses at the last-level cache divided by misses at the first—collapses on a two-level hierarchy to the fraction of L2 accesses that proceed to DRAM. LFMR $\approx$ 1: the L2 contributes nothing; the access stream is cache-defeating (PiM-favourable). LFMR $\leq$ 0.35: the L2 is absorbing reuse; near-data placement would forfeit a working cache.

4.3.3 Memory intensity (MPKI)

$$\text{MPKI} \;=\; \frac{\text{DRAM bytes}/32}{\text{warp instructions}/1000}, \tag{4.2}$$

i.e., DRAM 32-byte-sector fetches per thousand *warp* instructions. (The instruction denominator is warp-level, 32× smaller than thread-level; conflating the two is a classic factor-of-32 error the pipeline guards against in tests.)

4.3.4 Arithmetic intensity and the roofline

$$\mathrm{AI} = \frac{\mathrm{FLOPs}}{\mathrm{DRAM\ bytes}}, \qquad \mathrm{FLOPs} = \mathrm{adds} + \mathrm{muls} + 2 \times \mathrm{FMAs}. \tag{4.3}$$

Attainable performance is $\min(P_{\mathrm{peak}},\ \mathrm{AI} \times B_{\mathrm{peak}})$ with the *measured* ceilings of §4.2.2 [11]. A kernel under the sloped segment is memory-bound; under the flat segment, compute-bound. The FLOP numerator selects the dominant operand type automatically (FP32 for cuVSLAM; FP16/INT for other adapters' workloads), keeping the roofline meaningful across workload families.

4.3.5 Occupancy and the stall taxonomy

Occupancy is the achieved fraction of maximum resident warps; below 25% the latency-hiding mechanism of §2.1 is starved. When a warp cannot issue, the hardware records *why*; the classifier consumes the dominant reason: `long_scoreboard` (waiting on DRAM/L2 data—a memory stall), `short_scoreboard` (shared-memory data), `lg/mio/tex throttle` (memory-pipe queues full), `wait` (fixed-latency dependency chains—*not* memory), `barrier` (synchronization). The pair (dominant stall, occupancy) separates "memory-latency-bound" (G4) from "dependency-bound" (G7)—two conditions with identical symptoms in a time-only profile and opposite architectural remedies.

4.3.6 Coalescing

Sectors per request, from counters, with the trace-side check of sectors-per-warp-access from NVBit (§7.1); ≥ 8 marks a scattered gather.

4.3.7 Locality (trace-side)

Defined in §7.1: reuse-distance CDFs, footprints, and inter-launch overlap, computed from global-space addresses only.

Table 4.1: Classifier thresholds (values as frozen in `analysis/classify.py`).

threshold	value	meaning
`sol_hi`	40 %	a speed-of-light value counted as "high"
`sol_ratio`	1.5×	margin by which compute SoL must exceed memory SoL for compute-boundedness
`dram_sat`	50 %	DRAM SoL treated as bandwidth saturation
`lfmr_hi` / `lfmr_lo`	0.40 / 0.35	L2 "not helping" / "earning its keep"
`sect_scatter`	8	sectors/request marking a scattered gather
`occ_low` / `occ_low_dep`	25 % / 30 %	occupancy below which latency (G4) / dependency (G7) stalls cannot be hidden

4.4 The classifier and taxonomy

4.4.1 Thresholds

The decision tree's thresholds (Table 4.1) are stated once in the classifier source and imported live into all documentation and the dashboard, so prose and code cannot drift. They are calibrated on the characterization corpus and then *frozen*; robustness is established by the ±25% sensitivity stress (§4.4.3) and the known-truth calibration suite (§6.5) rather than by tuning.

4.4.2 The ordered rules

First matching rule assigns the class:

1. **G5 compute-bound**: compute SoL $\geq$ 40% and $\geq$ 1.5× memory SoL.
2. **G6 on-chip-bound**: a shared-memory/MIO stall dominates while DRAM is not saturated.
3. **G1 bandwidth-bound**: DRAM SoL $\geq$ 50%.
4. **G2 coalescing-bound**: memory-limited and sectors/request $\geq$ 8.
5. **G3 L2-reuse-bound**: memory-limited and LFMR $<$ 0.35.
6. **G4 latency-bound**: a memory stall dominates at occupancy $<$ 25% with DRAM unsaturated.
7. **G7 dependency-bound**: a dependency stall (`wait`, `barrier`, ...) dominates at low occupancy with neither memory nor compute high.
8. **G0 no-signal**: nothing dominant (tiny or launch-tax kernels; screened rather than architecturally interpreted).

Table 4.2: The GPU bottleneck taxonomy and its default near-data affinity.

class	meaning	PiM affinity
G0	no dominant signal / sub-floor	— (candidate for CPU)
G1	DRAM bandwidth saturated	**strong**
G2	scattered access wastes bandwidth	**conditional** (scatter PiM or layout fix)
G3	L2 reuse effective	weak (keep the cache)
G4	memory-latency stalls, low occupancy	strong if cache-defeating
G5	compute-bound	none (keep on GPU)
G6	on-chip/shared-memory-bound	none
G7	dependency stalls, memory idle	none (fix occupancy first)

Class G7 was not part of the initial hypothesis; it emerged from cuVSLAM kernels that stall on instruction dependencies at low occupancy with memory idle. That the taxonomy grew a class under pressure from data is by design —DAMOV's own classes were likewise derived, not decreed.

4.4.3 Sensitivity analysis

Every kernel is re-classified with all thresholds scaled by ±25%. A kernel whose class changes under this perturbation is flagged *borderline* and cannot carry high confidence into the verdict stage; every headline kernel in this thesis is threshold-stable. The observed flips concentrate exactly where physics predicts: at the L2-capacity crossover, where a working set close to 25 MB moves between G1 and G3.

4.5 From class to verdict: substrate and fault mapping

Class alone does not determine placement; the join with data-structure *persistence* (Chapter 8) does. Persistence takes three values, measured from the allocation journal and stage map: *streaming* (per-frame data, consumed once), *hot-persistent* (resident state reused every frame, e.g. the BA linear system), and *cold-persistent* (session-lifetime state touched episodically, e.g. the keyframe database). Table 4.3 gives the mapping.

When the verdict is "stay on GPU," the same evidence names the fault: spill-dominated traffic ⇒ register-pressure/compiler fault (the traffic is scratch, not data); high sectors/request ⇒ data-layout fault; dependency stalls at low occupancy ⇒ launch-configuration fault; a flat reuse CDF ⇒ structurally cache-immune (no cache-size remedy exists). This dual output—substrate candidacy *or* a named fix—is what makes the characterization actionable in both directions.

Table 4.3: Substrate verdict mapping.

condition	affinity	substrate
G1 ∧ streaming	strong	near-sensor SRAM (consume before DRAM)
G1 ∧ otherwise	strong	DRAM-PiM (near-bank)
G2	conditional	scatter-capable PiM, or data-layout fix first
G4 ∧ cache-defeating	strong	near-memory compute
cold-persistent ∧ large	strong	in/near-storage (ISP)
G3	weak	GPU (a larger cache wins)
G5 / G6 / G7	none	GPU
tiny (occ < 8%, < 1 ms)	—	CPU/host

4.6 The placement model

To bound the payoff of offloading without a simulator, a first-order analytical model: for a kernel of GPU time t with memory-bound fraction m, executed on a near-data substrate with internal-bandwidth advantage k and relative compute throughput c,

$$t_{\text{pim}} \;=\; t\,\frac{1-m}{c} \;+\; t\,\frac{m}{k}. \tag{4.4}$$

A kernel is offloaded only if its affinity is admitted by the scenario and $t_{\text{pim}} < t$. Two scenarios bound the claim: *conservative* (k=4, c=0.5, strong-affinity only) and *moderate* (k=8, c=0.75, strong + conditional). An energy variant applies the same split to the measured energy baseline. A standing rule constrains all of this: model-derived and (future) simulator-derived numbers are reported only as *deltas against the measured GPU baseline*, never as absolute performance claims.

4.7 DAMOV parity: the completeness argument

RQ1 asks whether every DAMOV component has a GPU analogue here. The correspondence, element by element:

- *Step 1, screening.* DAMOV: VTune top-down, keep functions with memory-bound share >30% and ≥3% of cycles. Here: nsys time-share plus the ncu SoL pair; sub-floor kernels are screened (G0), not classified.
- *Step 2, locality.* DAMOV: architecture-independent spatial and temporal locality from CPU address streams, clustered. Here: NVBit per-warp traces → reuse-distance CDFs (temporal) and sectors-per-warp coalescing (the GPU's spatial analogue), *memory-space filtered* so compiler scratch cannot pollute locality (§7.3).

- *Step 3, intervention.* DAMOV: simulated host / host+prefetch / NDP across a 1–256-core sweep, with classes defined by the response. Here, two measured axes replace the unavailable core sweep (a GPU grid is compiled into the launch; SM count is not variable on this part): (a) a *clock-domain sweep*—core and memory clocks scaled independently (1620→810 MHz core = 0.5× compute; 7001→5001 = 0.71× DRAM)—yields falsifiable per-class predictions (G1/G2 track the memory clock, G3/G5/G6/G7 the core clock, G4 neither strongly); (b) the *measured* reuse-distance hit-CDF answers the cache-capacity question directly, per kernel, from 64 KiB to 48 MiB, where DAMOV inferred it from aggregate sweeps. The third, simulated axis (NDP configurations for Accel-Sim) is generated from the verdicts and deferred to the follow-on study.

- *Classification.* DAMOV: six classes from a threshold tree over temporal locality, LFMR trend, MPKI, AI. Here: eight classes (G0–G7) from the ordered tree of §4.4; the deviations (a coalescing class; a G4/G7 occupancy split; no L3-contention class because the L2 is the last level) are each forced by a physical difference of the GPU.

- *Robustness.* DAMOV: frozen thresholds validated on 100 held-out functions (97% correct); cache-size sweeps; an independent clustering algorithm. Here: the ±25% sensitivity stress; a known-truth *calibration suite* of eight archetype kernels written to embody each class and classified blind (§6.5); cross-device agreement across two microarchitectures (§6.4); and *two* independent clustering algorithms (k-means and Ward hierarchical) recovering the taxonomy (§6.3).

Where DAMOV's breadth was 144 applications, this study is deliberately deep rather than wide—one production application, 49 kernels, 27 sequences, 192 configuration variants—with breadth provided architecturally: the harness's adapter interface runs arbitrary GPU workloads through the identical pipeline, and the calibration archetypes serve as the suite-artifact analogue.

Chapter 5

Measurement Validity: Accuracy, Neutrality, and Noise

A characterization inherits the validity of its measurements. This chapter establishes, in order: that the system under measurement was computing *correct* trajectories (§5.1); that observing it did not change its behaviour (§5.2); and that residual run-to-run variation is understood and bounded (§5.3). Together these answer RQ5 and constitute finding **F13** of the evidence chain.

5.1 Accuracy validation: the 141-configuration matrix

Every combination of dataset, sensor modality, and pipeline mode that the on-disk corpora support was materialized into a configuration—141 in total—and each was run to completion and scored against ground truth with the vendor's own metrics (RMSE APE after alignment; segment-relative RTE; rotational RE). A convergence gate (segment-relative translational drift below 5%, a scale-independent criterion valid from room-scale to kilometre-scale trajectories) separates runs that track from runs that diverge; every excluded run is itemized with a stated cause rather than silently dropped.

Table 5.1 compares the converged mode averages against the cuVSLAM technical report [2]. Two facts carry the chapter. First, *the modes used for the entire memory characterization—stereo and RGB-D SLAM—reproduce the published accuracy*: EuRoC stereo to within millimetres, TUM RGB-D better than published, and KITTI matching on the definition-stable per-segment drift metric (0.82% at the 500 m segment versus the 0.85% leaderboard figure; absolute APE over kilometre paths is scale-sensitive and reported as directional only). Second, every discrepancy in the remaining modes has a named, verified cause, none of which touches the characterized configurations.

Table 5.1: Converged mode-average accuracy vs. the cuVSLAM technical report (RMSE APE in metres, odometry/SLAM). The characterization uses the stereo and RGB-D modes.

mode	ours	published	verdict
EuRoC stereo	0.114 / 0.051	0.13 / 0.054	millimetre-level match
TUM RGB-D	0.060 / 0.047	0.11 / 0.065	better than published
ICL-NUIM (mono-depth)	0.099 / 0.136	0.026	same order (cm)
KITTI stereo	4.01 / 3.63	3.00 / 1.98	segment drift matches (0.82% vs 0.85%)
EuRoC stereo-inertial	0.401 / 0.328	0.19 / 0.13	under-tuned IMU (disclosed)

5.1.1 Discrepancies and their causes

Inertial mode is under-tuned—by configuration, not by cuVSLAM. Only a minority of EuRoC inertial runs converge, and—diagnostically— enabling the IMU makes accuracy *worse* than pure stereo on the same sequences. That inversion is the classic signature of mis-scaled IMU noise densities or camera–IMU extrinsics: the estimator over-trusts a mis-modelled sensor. The vendor calibrates these per dataset; the matrix used a generic inertial configuration. The defect is confined to a mode the memory characterization never uses, and the remedy (populating per-sequence IMU parameters from the datasets' calibration files) is configuration work, tracked in the roadmap.

The hardest EuRoC sequence diverges in every mode. V2_03 combines aggressive motion with heavy motion blur; the vendor's own report publishes it only under stereo-inertial and excludes it from stereo averages. The same exclusion is applied here, with the sequence retained in per-sequence tables.

TUM-VI raw fisheye is invalid input, not a tracking failure. TUM-VI's ∼195° fisheye exceeds the pinhole-undistortion field of view the library supports (<180°); feeding raw fisheye produces kilometre-scale divergence deterministically. The documented preparation (undistort to pinhole with edge masks) was identified in the dataset notes; runs without it are classified invalid-input and excluded.

Monocular needs scale-free alignment. Monocular SLAM recovers shape but not metric scale; scoring it with SE(3) alignment conflates scale error with drift. Monocular rows are therefore excluded from SE(3) comparisons pending Sim(3) evaluation [16]—an evaluator-metric choice, not a tracking result.

5.2 Profiling is accuracy-neutral

The deepest reflexivity threat is that instrumentation changes the system it observes. Three experiments close it.

Instrumented build equals baseline build (QoR). The TaggedAllocator instrumentation (Chapter 8) lives in a rebuilt cuVSLAM binary. Re-running representative configu-

rations on both wheels shows the instrumented build's trajectories equal the baseline's—bit-identical on EuRoC configurations, and within run-to-run nondeterminism elsewhere (largest observed difference 0.24 m over a kilometre-scale KITTI path, within the sequence's own re-run scatter; see §5.3). The journaling itself is gated by an environment variable and touches only allocation paths, which execute at initialization, not per frame.

Each profiler is trajectory-neutral. Running the same configuration plain and under each instrument shows: nsys and ncu produce *bit-identical* trajectories on deterministic (synchronous) modes, and NVBit-traced runs agree within ∼2 mm APE—the instrumentation slows execution greatly but does not alter the computation.

Neutrality holds across the full feature matrix. A 192-variant campaign (every accuracy configuration profiled as-is, plus finer feature toggles—sync/async/CPU SLAM, motion-model and denoising toggles, landmark export, multi-camera precision, unrectified input, depth-stereo tracking—on representative sequences per modality) compares profiled against plain APE with tolerance max(5 cm, 5%): **166/192 pass outright**, with 121 bit-identical. Every one of the 26 flagged variants was investigated and traced to a benign cause—dominated by the nondeterministic scatter of long loop-closing sequences (§5.3) and by toggles that legitimately change the computation (e.g. CPU-SLAM changes the back-end, hence the trajectory, independent of profiling); a re-check against plain re-runs disproved the one suspected genuine interference case.

5.3 Nondeterminism, characterized

GPU SLAM is not perfectly repeatable: floating-point reductions reorder across runs, and asynchronous modes add scheduling freedom. Rather than averaging this away, the campaign measured it. On short and mid-length sequences, repeated runs are bit-identical in synchronous modes. On long loop-closing sequences (kitti00 is the canonical case) the final trajectory is *bimodal*: loop-closure acceptance is threshold-sensitive, and a borderline loop either fires or does not, moving whole-path APE by tenths of a metre. The decisive control: *plain, uninstrumented re-runs scatter by the same amount and between the same modes.* The scatter is a property of the workload, not of the profilers—which is itself a characterization result a hardware study must respect (single-run accuracy comparisons on such sequences are meaningless at sub-scatter resolution).

5.4 Summary of the trust chain

The system under measurement reproduces its vendor's published accuracy in the characterized modes (141-run matrix); the instrumentation is output-neutral (bit-identical where the workload is deterministic, and within measured workload scatter elsewhere, across 192

variants); the noise floor of all timing quantities is CoV 0.14% at locked clocks (§4.2.2); and every classification threshold is sensitivity-stressed (§4.4.3). Every number in Chapters 6–9 rests on this chain.

Chapter 6

Kernel-Level Characterization

This chapter reports the classification of cuVSLAM's 49 GPU kernels across the 27-sequence corpus and then subjects that classification to four independent validations: input stability (§6.2), unsupervised recovery by two clustering algorithms (§6.3), cross-microarchitecture agreement (§6.4), and a known-truth calibration suite (§6.5). It closes with the system-edge measurement of host–device transfers (§6.7).

6.1 What the screen reveals

The nsys timeline over full sequences shows the expected shape of a frame-synchronous perception pipeline: a small set of front-end kernels launched every frame (image cast, Gaussian pyramid scaling, gradient convolutions, Lucas–Kanade tracking, and the matcher pair) accounts for the bulk of launches (∼80 per frame; ∼200,000 launches over a 2,500-frame sequence), with the bundle-adjustment solver family (`sba::*`, plus cuSOLVER helper kernels) forming a second tier, and the SLAM-layer keyframe kernels (`st_build_cache`, `st_track_with_cache`) launching sparsely—per keyframe and per loop-closure scan respectively. The NVTX stage table (measured, not name-matched; §2.6) confirms stage membership: `st_track_with_cache` executes inside the loop-closure-and-optimization range, `st_build_cache` inside keyframe ingestion—the measured anchor for every loop-closure claim that follows.

6.2 Class stability across inputs

The classifier of §4.4 was applied independently per sequence, then compared across the corpus. The headline: **91% modal class consistency**—24 of 49 kernels receive the identical class in every one of the 27 sequences, and 42 of 49 agree in at least 80% of them. The bottleneck class is, to first order, *a property of the kernel, not of the input*.

The residual flips are not noise; they are physics. They concentrate on kernels whose

working set sits near the 25 MB L2 capacity: on small indoor sequences the set fits (L2-reuse behaviour, G3), on street-scale sequences it spills (bandwidth behaviour, G1). The sensitivity analysis (§4.4.3) flags exactly these kernels as borderline, and the substrate synthesis (Chapter 9) treats their verdicts as workload-conditional—25 of 49 verdicts flip across workload scale, and the flip direction is always the physically expected one.

6.3 The taxonomy is discovered, not asserted

The decision tree assigns labels; whether the *classes themselves* are real structure in the data is tested without labels. Pooling every kernel's feature vector (memory/compute/DRAM speed-of-light, LFMR, occupancy, sectors-per-request, and the two dominant stall shares) across sequences and running k-means over a sweep of cluster counts, the silhouette criterion selects $k = 7$–8—matching the seven populated classes plus the screened remainder—with cluster purity 0.68 and adjusted Rand index 0.30 against the tree's labels. An entirely different algorithm—Ward hierarchical clustering [33], cut at k=8 over the pooled cloud—reproduces the same agreement: **ARI 0.31, purity 0.675**. Two unrelated algorithms, given no thresholds and no labels, find the same ∼8-way structure the tree encodes. The tree should be read as the *labelling* of natural structure, and the clustering as its *independent validation*—precisely DAMOV's own discipline, reproduced on GPU data.

6.4 Cross-microarchitecture agreement

If the classes captured artifacts of one GPU, they would not survive a change of microarchitecture. The same workload (TUM office) was characterized independently on two deliberately different devices: the `sm_89` workstation instrument and an `sm_75` laptop part with a quarter the memory and a third the power budget. **36 of 45 signal kernels (80%) receive the same class on both** (76.6% including the launch-tax G0 tier). The honest caveat is stated with the number: the two devices differ in memory system as well as core generation, so some disagreements are physically real L2-capacity crossovers rather than classification error—making 80% a conservative floor for the device-independence of the classification. This is the GPU analogue of DAMOV's core-type-independence check, under a harsher confound.

6.5 The calibration suite: classifying known truth

DAMOV validated frozen thresholds on held-out functions with known behaviour. The analogue here is a *calibration suite*: eight archetype kernels written so that their ground-truth class is known by construction—a streaming triad (G1), a random gather (G2), an L2-resident

sweep (G3), a coalesced pointer-chase (G4), an FMA polynomial (G5), a bank-conflicted shared-memory kernel (G6), a single-warp dependency chain (G7), and a sub-screen tiny kernel (G0)—compiled and pushed through the identical capture-and-classify pipeline, blind. The first pass classified five of eight as designed; analysis of the three misses showed each to be a defect in the *archetype*, not the classifier: a per-lane-random pointer-chase is legitimately both scattered and latency-bound (G2+G4); a barrier-heavy shared-memory kernel legitimately presents as dependency-stalled rather than G6; and a sub-floor kernel is legitimately *screened*, per the pipeline's own Step-1 semantics. Each archetype was corrected to embody a single class—*with the classifier untouched throughout*—and the suite is retained in the artifact as a permanent regression harness: any future change to thresholds or metrics must re-classify known truth correctly before it can land.

6.6 Near-data affinity of the workload

Rolling the per-kernel classes up by measured GPU time: across the 27-sequence campaign, **21% of GPU time is in strong-affinity classes, 40% in conditional-affinity classes, and 28% in no-affinity classes** (remainder sub-floor)—i.e. roughly **61% of GPU time carries near-data affinity** under the taxonomy's default mapping, rising to ∼72% time-weighted in the deep single-sequence studies where the loop-closure path is exercised densely. This is the quantity the placement model of Chapter 9 prices; it is reported here with its composition rather than as a headline slogan, because the conditional 40% is exactly the portion whose verdict depends on the scatter-PiM-vs-layout-fix question that the locality and attribution chapters resolve.

6.7 The system edge: host–device transfers

Independent of any kernel's internals, the timeline measures the pipeline's boundary traffic: explicit host-to-device and device-to-host copies cost **41% of total kernel time** on the RGB-D workload, and the per-frame sensor upload is **1.68 MB/frame**—the camera image and depth crossing PCIe into GPU DRAM before any kernel touches them. This is the single most direct near-sensor argument in the thesis: it is traffic that exists only because the pixels are *born* on the wrong side of the bus, and no on-GPU optimization can remove it. The host-side companion measurements (66 MB of storage ingestion per run; 708 MB peak host memory) appear in the synthesis chapter, where the boundary picture is assembled end-to-end.

Chapter 7

Machine-Independent Locality Analysis

Counters summarize; address traces prove. This chapter reports the DAMOV-style locality analysis computed from NVBit per-warp address traces: the metrics (§7.1), the front-end result (§7.2), the loop-closure-scan result and the self-correction that produced it (§7.3), and the session-scale growth measurement (§7.4).

7.1 Trace-side metrics

From each kernel's global-space address stream, per launch:

Footprint — the number of distinct 32-byte sectors touched, ×32 B: the exact working-set size, measured rather than inferred from traffic.

Reuse-distance CDF — for each access, the count of distinct sectors touched since that sector's previous access (LRU stack distance [21]); the CDF evaluated at capacities from 64 KiB to 48 MiB *is* the predicted hit rate of any cache of that size, independent of any real cache's policy details. A flat curve means no cache size helps: the misses are compulsory.

Coalescing, trace-exact — sectors touched per warp access (1–32), plus active lanes per access, separating true scatter from divergence-induced apparent scatter.

Inter-launch overlap — the Jaccard similarity $|A \cap B|/|A \cup B|$ between consecutive launches' footprints: does a recurring kernel re-touch the same data (cacheable across launches) or migrate?

One rule governs all of these: *memory-space filtering*. Only global-space instructions (`LDG`/`STG`) enter locality analysis; shared-memory (`LDS`/`STS`) and local-memory (`LDL`/`STL`, register spill) records are bucketed separately. This rule was not a precaution added in advance—it was forced by the measurement failure documented in §7.3, and is now the methodology's most exportable lesson.

7.2 The front-end is cache-immune streaming

For every per-frame front-end kernel (image cast, pyramid scaling, gradient convolutions, LK tracking), the reuse-distance CDF is **flat from 64 KiB to 48 MiB**: the hit rate a 64 KiB cache would achieve is the hit rate a 48 MiB cache would achieve, because all reuse is at tiny distances (caught by any cache) and everything else is a first touch of streaming pixel data (caught by none). Global-space accesses run at ∼4.0 sectors per warp access, fully coalesced, at 32.0 active lanes.

The architectural reading is sharp: *no realizable cache helps this tier*—its misses are compulsory. Bigger caches are the wrong axis entirely; the only remedies are moving fewer bytes (algorithmic) or consuming the stream before it reaches DRAM (near-sensor placement, §6.7). An early counter-only reading had suggested one convolution kernel was a scattered "data-layout target" (15.4 sectors per access) with high reuse; space filtering dissolved both signals—they were shared-memory tile traffic, on-chip and irrelevant to DRAM. After the correction, the front-end is *uniformly* stream-clean, and the correct optimization target is on-chip, not layout.

7.3 The loop-closure scan: a self-correction on record

The loop-closure scan kernel `st_track_with_cache` produced the project's most instructive measurement arc, reported here in full because its methodological value exceeds that of a clean result.

Round one: counters vs. trace. Nsight Compute counters read the kernel as a scattered gather—18–30 sectors per request. The first NVBit trace analysis said the opposite: 2.1 sectors per warp access, 99.4% of accesses touching ≤4 sectors—apparently tightly coalesced streaming. The trace being nominally ground truth, the project's interim report concluded the counter metric was a proxy artifact—"the trace overturns the counters."

Round two: the join exposes the confound. When the attribution join (Chapter 8) later matched trace addresses against live allocations, 88–98% of most kernels' trace records matched *no allocation at all*—impossible for real data traffic. The explanation: the tracer records *every* memory space, and for `st_track_with_cache`, **92.6% of all recorded accesses were register-spill traffic** in local space— which is *perfectly coalesced by construction* (§2.2)—drowning the actual data accesses in the blended statistics.

Round three: space-filtered re-derivation. Filtering to global space only and re-deriving every table: `st_track_with_cache`'s *data* accesses are **23.4–30.0 sectors per warp access, with only 2.3–6.0% of accesses touching ≤4 sectors**—a scattered gather, *exactly what the counters had said*. The interim conclusion was withdrawn in a dated, in-place correction; the classifier's G2 label stands; and the two instruments now agree. Table 7.1 juxtaposes the blended and corrected views.

Table 7.1: The `st_track_with_cache` correction: blended-space analysis vs. space-filtered re-derivation (TUM room-scale / KITTI street-scale).

metric	blended (all spaces)	global-only	local-only (spill)
sectors / warp access	2.1	23.4 / 30.0	2.0
accesses ≤4 sectors	99.4 %	6.0 / 2.3 %	100 %
share of recorded accesses	100 %	∼5–7 %	∼92.6 %
per-scan footprint (MB)	0.467 / 1.096	0.464 / 1.093	∼0.003
inter-launch Jaccard	0.67 / 0.90	0.67 / 0.90	1.0

Three durable lessons. *Methodological*: unfiltered GPU address traces are dominated by spill and scratchpad records; any locality or attribution analysis that does not filter by memory space is unsound—a warning directly applicable to other groups using binary-instrumentation traces. *Evidential*: two independent instruments agreeing *after* a documented reconciliation is stronger support than either alone; the disagreement was the pipeline catching its own error. *Architectural*: the kernel's DRAM volume and its DRAM *pattern* have different owners —volume is dominated by compiler spill (a register-file/occupancy trade, fixable on-GPU), while the data-side pattern is a genuine scattered gather over the keyframe descriptors (a scatter-PiM or layout question). Only the attribution of Chapter 8 could have separated them.

7.4 Session-scale growth and migration

Two properties of the scan survive the correction untouched (the spill window is only ∼3 KB of the footprint). First, the per-scan working set *grows with the map*: 0.46 MB per scan at room scale (TUM) to 1.09 MB at street scale (KITTI). Second, it *migrates*: consecutive scans overlap with Jaccard 0.67 (room) to 0.90 (street)—10–33% of the touched set turns over between scans. Each individual scan is nearly L2-resident; the union over a session—the entire keyframe database, scanned incrementally, growing without bound with distance travelled—is what no cache holds. This is the measured locality half of the in-storage case; the allocation half (the database's GPU-resident state is a fixed buffer while the growing store lives host-side) is established in Chapter 8.

Chapter 8

Data-Structure Attribution: Naming Every Byte

A kernel-level claim ("this kernel is memory-bound") is not yet an architect's claim ("the keyframe database belongs near storage"). Closing that gap requires knowing, for each byte of DRAM traffic, *which named data structure it belongs to*. This chapter presents the TaggedAllocator attribution: the instrument (§8.1), the coverage discipline (§8.2), the static-budget finding (§8.3), the per-kernel attribution results (§8.4), the register-spill finding (§8.5), and the bounded residuals (§8.6).

8.1 The three-layer instrument

Attribution is a chain of fragile assumptions (address → allocation → owning code → data-structure name), so the design uses three independent layers that cross-check:

Layer 1 — source-side journal (the semantic layer). Every GPU allocation in cuVSLAM flows through a handful of allocation wrappers; an injected journal records, for each allocation and free, the pointer, size, wrapper kind, and a host *call stack*. Resolved offline against debug symbols, the innermost non-plumbing frame names the owning data structure (e.g. the Schur-complement solver's constructor ⇒ `ba_linear_system`); a fixed tag vocabulary (`images_raw`, `pyramid_levels`, `feature_tracks`, `ba_linear_system`, `keyframe_descriptors`, …) keeps names stable across runs and sequences. The journal is gated by an environment variable; disabled, the instrumented binary is behaviourally identical to the shipping one, and its *output* equivalence is proven in §5.2. Pinned host buffers (device-visible under unified addressing) are journaled with a distinguishing suffix, since their traffic crosses PCIe rather than the DRAM bus.

Layer 2 — driver-side sidecar (the temporal layer). The NVBit tool independently logs every driver-level allocation and free, stamped with the current kernel-launch counter—the same clock the address trace uses. This provides allocation *lifetimes* in trace time

(essential when an address is freed and reused by a different structure, which the test suite exercises) and catches allocations that bypass the wrappers (linked math libraries' internal scratch), which are tagged as driver-internal rather than silently mislabeled.

Layer 3 — the join. A streaming pass over the address trace maintains the live-allocation set in launch order and resolves each global-space access to its containing allocation's tag; shared-space and local-space records are bucketed as on-chip and spill respectively (§7.3); anything unresolved lands in an explicit `unmapped` bucket—the join's own error bar, reported, bounded, and investigated rather than hidden.

8.2 Coverage: auditing the windows

Traces are captured in windows, and sparse kernels can drift out of a planned window between runs (launch indices shift with keyframe timing). The pipeline therefore treats coverage as a proof obligation: after each campaign, an audit diffs every sequence's *full* launch map (from a cheap uninstrumented pass) against the union of attributed kernels, and a planner captures gap-fill windows anchored on *dense clusters* of the missing kernels' launches (isolated occurrences drift; clusters do not). Across the 27-sequence attribution campaign this loop converged to **zero missing kernels**, with the audit artifacts committed. Without this discipline, three sequences would have silently lacked the loop-closure scan and one the keyframe-refresh cluster—precisely the architecturally interesting kernels.

8.3 The GPU memory budget is static

The journal's first result is structural. Over a full 2,500-frame loop-closing session, cuVSLAM performs **240 device allocations totalling 108.65 MB—all at initialization, none freed mid-run—and the identical 240 in a 30-frame run**. The budget by tag (Table 8.1): images and pyramids ∼71 MB, the BA linear system 24.3 MB (with a 15.4 MB pinned-host mirror), and—the architecturally decisive row—**a fixed 6.7 MB keyframe-descriptor buffer**. The keyframe *database*, the only data structure that grows with session length, does not grow on the GPU at all: its growing store lives host-side in the memory-mapped database, confirmed by direct host measurement (708 MB peak host memory against the static GPU budget; §9.5). The in-storage verdict of Chapter 9 rests on this measured division of labour, not on inference from kernel behaviour.

8.4 Attribution results

The join resolves **274 of 274** journaled allocations (240 device + 34 pinned-host) with zero unknowns, and across the full 27-sequence campaign, **48 of 49 kernels receive a**

Table 8.1: The static GPU allocation budget (full session; peak = total, nothing freed mid-run).

tag	allocations	peak live (MB)
`images_raw`	92	39.97
`pyramid_levels`	99	31.15
`ba_linear_system`	31	24.26
`keyframe_descriptors`	2	**6.73 (fixed)**
`feature_tracks`	8	4.27
`depth_pyramid`	3	2.21
`icp_state`	4	0.07
device total	240	108.65
pinned host (device-visible)	34	17.14

unanimous top data-structure tag in every sequence they appear in—the attribution analogue of the 91% class consistency of §6.2, and the license to make per-kernel data-structure claims input-independently. Selected rows (steady-state window; shares are sector-weighted):

- `st_track_with_cache` (loop-closure scan): **92.6% register spill**, with the global remainder on `keyframe_descriptors`; unmapped <1%.
- `st_build_cache` (keyframe ingest): 93% `keyframe_descriptors`.
- `sba::reduced_system_stage_2` (solver core): **96.9%** `ba_linear_system`—one named, pre-sized, contiguous structure carrying essentially all of the solver's DRAM traffic.
- Front-end compute kernels (convolutions, feature selection, sorting): 83–98% of accesses are shared-memory tiles (on-chip; no DRAM claim), with the global residue falling exactly on the pyramid, image, and track structures the pipeline stage implies.

Mode differences surface exactly where they should: stereo configurations journal no depth or ICP structures; the loop-free KITTI sequences show `st_track_with_cache` launches only where their trajectories genuinely close no loops—the attribution reproduces the workload's semantics, which is itself a correctness check.

8.5 The register-spill finding

The single most consequential attribution row is `st_track_with_cache`'s: the kernel that motivates the near-data discussion spends **92.6% of its DRAM volume on register spill**—compiler-generated scratch traffic, invisible as such to every counter-only methodology, which would (and initially did) book it as data traffic. The finding cuts twice. For *this* workload: the loop-closure scan's bandwidth bill is mostly a register-pressure artifact (its

per-thread descriptor state exceeds the register budget), so a register-file/occupancy trade or spill-aware compilation attacks the volume, while the residual—the true database gather (§7.3)—is the scatter-PiM candidate. For the *methodology*: spill is a first-class DRAM citizen that per-space attribution alone can expose; a scope caveat accompanies it (register allocation is a codegen decision of this compiler for this target; the taxonomy transfers, the spill *quantity* must be re-derived per target, and its cross-target validation is scheduled work).

8.6 Bounded residuals

The join's honesty budget: for most kernels the unmapped share is below 1–7%. Two systematic residuals remain, both bounded and explained. First, one solver-assembly kernel carries ∼40% unmapped traffic *identically across all 27 sequences*—the signature of static module memory (device-side globals allocated by the module loader, invisible to both journal layers by construction), not of a lurking data structure; a committed analysis names the two kernels (`sba::build_full_system`, `lk_track`) that carry 83% of all unmapped traffic, turning the residual into a prioritized target list for the planned kernel-argument correlation layer. Second, texture-path reads (the LK tracker samples pyramids through texture units) never appear in the address trace, so image-structure traffic is a stated lower bound, with counter-side texture totals corroborating the affected kernels. Neither residual moves any verdict: the affected kernels' classes rest on counters and stage evidence that do not depend on the unmapped share.

Chapter 9

Synthesis: Substrate Verdicts and Measured Baselines

This chapter assembles the evidence of Chapters 6–8 into the thesis's deliverable: a per-tier substrate placement with the measurement behind each cell (§9.1), the dynamics of the verdicts across workloads (§9.2), the analytical placement bound (§9.3), and the measured energy and host-side baselines that a follow-on architecture study must beat (§9.4, §9.5).

9.1 The three-way split

cuVSLAM's memory behaviour resolves into three tiers, each with a different substrate answer (Table 9.1).

The split is not a metaphor; each row is the conjunction of an NVTX-anchored stage, a bottleneck class, a locality curve, and a named allocation. And the rows are *different*—which is the finding. A single "SLAM accelerator" would be aimed at three mutually incompatible targets; the measured answer is heterogeneous placement.

9.2 Verdict dynamics across workloads

Applying the verdict mapping (§4.5) per sequence: 24 of 49 kernels receive the same verdict unanimously; **25 of 49 flip with workload scale**, and the flips are driven almost entirely by DRAM speed-of-light crossing its saturation threshold as working sets grow from room to street scale—the physically expected driver, not classifier noise. Time-weighted, **∼72% of GPU time is offload-eligible in the deep studies** (61% across the breadth campaign; §6.6). The practical consequence for a system architect: placement for this workload class is *scale-conditional*, and a deployment at street scale crosses into near-data-favourable territory that a desk-scale evaluation would never reveal.

Table 9.1: The three-way substrate split, with the measurement behind each claim.

tier	measured evidence	substrate consequence
Streaming front-end (images, pyramids, gradients, tracking)	reuse-CDF flat 64 KiB→48 MiB (compulsory misses); fully coalesced at ∼4 sectors/access; 1.68 MB/frame sensor upload; 41% of kernel time in PCIe copies	**near-sensor SRAM / ISP**: consume pixels before DRAM; caches provably cannot help; the upload disappears by construction
Hot-persistent solver (BA linear system)	96.9% of solver-core traffic on one pre-sized 24.3 MB structure (+15.4 MB pinned mirror); compute-leaning classes; L2-effective where resident	**keep on GPU** at this scale (bank-level PiM streaming is the candidate if problem scale outgrows L2)
Cold-persistent back-end (keyframe database)	GPU-resident state fixed at 6.7 MB while the store grows host-side (708 MB peak host RSS); scan is a scattered gather (23–30 sectors/access) whose footprint grows 0.46→1.09 MB room→street with 10–33% turnover per scan; 92.6% of scan DRAM volume is register spill	**in/near-storage** for the growing store; **scatter-capable PiM or layout fix** for the resident gather; **register-file/spill SRAM** for the volume

9.3 The analytical placement bound

Pricing the eligible time with the placement model of §4.6: under the *conservative* scenario (k=4, c=0.5, strong-affinity only), **16 of 49 kernels** clear the offload bar; under the *moderate* scenario (k=8, c=0.75, strong + conditional), **26 of 49**. These are candidacy bounds, not performance claims: the model's k and c are scenario parameters, and per the standing rule, the follow-on simulation reports deltas against the measured baseline rather than absolutes. What the bound establishes is that the opportunity survives pessimistic assumptions—the offload set is not empty even at k=4 with half-rate compute.

9.4 The measured energy baseline

Whole-run GPU energy, measured by integrating sampled board power at locked clocks: **34.67 J for a representative sequence (12.54 W mean, 25.08 W peak)**. The number matters less as a result than as an anchor: "PiM saves energy" claims in the follow-on study will be expressed as deltas against this measured joule count, closing the loop the thesis's energy motivation opened. At robot power budgets, the mean measured draw is

itself a datum—the perception stack's GPU share is a two-digit-watt load whose largest single component, per this characterization, is data movement.

9.5 The host-side boundary

Completing the end-to-end picture from the host side: per run, **66 MB is read from storage** (the sensor-data ingestion that becomes the 1.68 MB/frame H2D upload of §6.7), and host memory peaks at **708 MB**—the keyframe/landmark store and its memory-mapped database—while the GPU allocation stays static at 108.65 MB (§8.3). The cold tier's data path is thus fully measured across its whole journey: storage → host memory → (a fixed GPU staging buffer) → a scattered on-GPU scan. Each arrow is a candidate for a different intervention (near-storage filtering, host-memory-resident scan, scatter-PiM), and the follow-on study inherits measured traffic on every one of them.

9.6 Faults named for the kernels that stay

For the majority of kernels the verdict is "stay on GPU," and the same evidence names each one's actionable fault: the loop-closure scan's register-pressure spill (a compilation/occupancy trade quantified at 92.6% of its DRAM volume); dependency-stalled low-occupancy kernels (G7) whose remedy is launch-shape, not memory; scatter-pattern kernels whose layout could be fixed in software before any hardware is proposed; and cache-effective kernels (G3) for which the L2 is demonstrably earning its keep and must not be traded away in an NDP design. A characterization that can argue *against* near-data placement, kernel by kernel, is the same instrument that makes its positive verdicts credible.

Chapter 10

Threats to Validity and Limitations

This chapter enumerates what could be wrong, what is deliberately out of scope, and what mitigation exists for each. The discipline follows DAMOV's own practice of publishing its limitations alongside its claims.

10.1 Internal validity

Instrumentation reactivity. Addressed head-on in Chapter 5: profilers are trajectory-neutral (bit-identical on deterministic modes across 192 variants), the instrumented build equals the baseline build, and traces are never used for timing. Residual risk: counter collection via kernel replay perturbs caches between replays; the methodology brackets cold- and warm-cache readings rather than trusting either extreme.

Threshold choice. The classifier's thresholds are frozen and sensitivity-stressed at ±25%; borderline kernels are flagged and denied high confidence. Two label-free clustering algorithms and a known-truth calibration suite independently support the class structure. Residual risk: the calibration suite's first pass exposed three archetype-design conflations (each legitimately spanning two classes); the archetypes were corrected with the classifier untouched, and the suite is retained as a regression gate. The designed clock-domain intervention (§4.7) has its per-class predictions stated ex ante and is scripted in the artifact; its execution completes the validation suite.

Attribution soundness. Three independent layers reconcile (274/274 allocations resolved); an explicit `unmapped` bucket bounds what the join cannot name (<1–7% for most kernels); the two systematic residuals (static module memory; texture-path reads) are named, quantified (83% of unmapped traffic in two kernels), and shown not to carry any verdict.

Nondeterminism. Long loop-closing sequences are measurably bimodal run-to-run; plain re-runs scatter identically to profiled runs, so the scatter is workload-intrinsic. All timing quantities carry a measured noise floor (CoV 0.14%).

10.2 External validity

One workload family. cuVSLAM is a single system, argued representative by production deployment rather than by breadth across SLAM implementations. Mitigations: the pipeline structure the taxonomy attaches to (streaming front-end, windowed solver, growing episodic database) is shared by keyframe-based SLAM generally; and the harness's adapter mechanism runs arbitrary GPU workloads through the identical pipeline, so breadth is an execution cost, not new methodology. Claims are scoped to the measured system throughout.

One primary GPU; codegen-dependent quantities. The classification survives a two-microarchitecture check at 80% agreement under a memory- system confound. However, the register-spill share (92.6% of the loop-closure scan's DRAM volume) is a property of this compiler targeting this architecture: register allocation changes with target and compiler version. The thesis scopes the spill *quantity* to the measured target, claims only the *phenomenon and the method* as general, and schedules the embedded-target re-derivation (Jetson-class hardware; the harness already targets it) as the first item of future work.

Dataset realism. Public benchmark corpora underrepresent degenerate lighting, dynamic scenes, and multi-camera rigs. The vendor's own multi-camera evaluation datasets are partly proprietary; the configuration regime covers the modes, not every deployment condition.

10.3 Construct validity

"PiM affinity" is candidacy, not benefit. The 61–72% offload-eligible time and the 16/49–26/49 placement counts are bounds from a first-order model with scenario parameters; no simulated or silicon NDP result is claimed. The standing deltas-only rule exists precisely to keep this line uncrossed until the follow-on study.

Metric proxies. Counter metrics are proxies with known artifacts —this thesis documents a case where a proxy was right and the naive ground-truth reading was wrong (§7.3). The defense against proxy error is redundancy: every headline claim rests on at least two instruments that agree after reconciliation.

Texture-path blindness. The address tracer does not observe texture-unit fetches; image-tier traffic is a stated lower bound, corroborated by counter-side totals. No verdict depends on the unobserved portion.

10.4 What is out of scope, and why

Simulation of the proposed substrates (the architecture study; its configurations are generated, its baselines measured here); a second SLAM system (scope discipline–breadth via

adapters instead); multi-tenant GPU sharing (cuVSLAM deploys single-tenant); CPU-side DAMOV replication (the origin methodology, not the contribution); and energy attribution below whole-run granularity (requires the simulator's power model; the measured whole-run joule count is the anchor).

Chapter 11

Conclusion and Future Work

11.1 What was established

This thesis set out to determine, by measurement, where a production visual-SLAM workload's computation belongs on the emerging spectrum of memory-centric hardware. Its answers, in the order of the research questions:

RQ1 — the methodology exists. GPU-DAMOV re-derives every component of the reference data-movement methodology for GPUs—screening, machine-independent locality, threshold classification, and the full robustness battery—with the three GPU-forced redesigns (two-level LFMR, occupancy-conditioned latency classes, a coalescing axis) and with DAMOV's simulated intervention replaced by measured ones. The parity argument is componentwise and complete at DAMOV's granularity (§4.7).

RQ2 — the classes are real and stable. Eight bottleneck classes (G0–G7, one of them—dependency-bound G7—forced into existence by the data) classify 49 kernels with 91% cross-sequence consistency, survive ±25% threshold stress, are recovered by two label-free clustering algorithms (purity ≈0.68 both), agree 80% across two GPU microarchitectures, and correctly classify a known-truth calibration suite after three archetype defects were fixed with the classifier untouched.

RQ3 — every byte has a name. The TaggedAllocator attribution resolves all 274 allocations and gives 48/49 kernels a unanimous data-structure tag across the corpus. Its two decisive findings: the GPU memory budget is *static* (the growing keyframe database lives host-side, its GPU-resident state a fixed 6.7 MB), and 92.6% of the loop-closure scan's DRAM volume is register-spill scratch—a compiler artifact no counter-only study could have separated from data traffic.

RQ4 — the placement. The workload splits three ways: a cache-immune streaming front-end (near-sensor placement; no cache size helps, measured); a hot-persistent solver that should stay on the GPU at this scale; and a cold-persistent, session-growing database tier whose scan is a genuine scattered gather (in/near-storage plus scatter-capable PiM can-

didacy). Time-weighted, 61–72% of GPU time carries near-data affinity; 16–26 of 49 kernels clear an analytical offload bar under conservative-to- moderate scenarios; and the baselines a follow-on study must beat are now measured: 34.67 J whole-run energy, 1.68 MB/frame sensor upload, 41% of kernel time in boundary copies, 66 MB storage ingestion, 708 MB peak host memory.

RQ5 — the measurements are trustworthy. The characterized system reproduces its vendor's published accuracy (141-configuration matrix); profiling is output-neutral (bit-identical trajectories on deterministic modes across 192 variants); the noise floor is measured (CoV 0.14%); and the project's two self-corrections—most notably the reversal of an interim "coalesced streaming" conclusion once register-spill traffic was separated from data—are documented in place, ending with independent instruments in agreement.

11.2 The thesis, restated

For Physical-AI perception workloads, the question "should we build PiM?" is ill-posed; the well-posed question is "*which data structure*, touched by *which kernel*, under *which access pattern*, belongs on *which substrate*?"—and it is answerable today, on production software, with commodity instruments, to the granularity of a named allocation and with quantified confidence. Answered for cuVSLAM, it yields not one accelerator but a placement: sensor-side consumption for streams, the GPU for the solver, near-storage and scatter-capable near-memory engines for the map—with the measured evidence attached to every cell.

11.3 Future work

The project's phased roadmap continues past this document:

1. **Complete the validation suite.** Execute the scripted clock-domain intervention (per-class predictions are stated ex ante: bandwidth classes track the memory clock, compute/on-chip/dependency classes the core clock) and fold its verdicts into the artifact.

2. **Name the last bytes.** The kernel-argument correlation layer (Layer 3) resolves the two named residual kernels carrying 83% of unmapped traffic; its target list and join interface are already committed.

3. **Embedded-target re-derivation.** Re-run the regime on Jetson-class hardware (the harness already drives remote targets) to re-derive the codegen-dependent quantities—above all the spill share—and test the taxonomy's transfer to the deployment-class part.

4. **The architecture study.** Feed the traces to Accel-Sim under the generated NDP configurations [31], with AccelWattch [32] for per-class energy, reporting speedup and energy *deltas* against this thesis's measured baselines —per the standing rule, never absolutes. The conservative/moderate placement scenarios of §9.3 become simulated designs; the 16–26-kernel offload sets are the experiment list.

5. **Breadth via adapters.** Run non-SLAM GPU workloads (DNN inference, GPU database operators) through the unchanged pipeline to test the methodology's generality claim—the adapter interface exists precisely so that breadth costs execution time, not engineering.

The characterization is complete; the design space it opens is measured, bounded, and—for the first time for a production Physical-AI perception stack—named, byte by byte.

Bibliography

[1] G. F. Oliveira, J. Gómez-Luna, L. Orosa, S. Ghose, N. Vijaykumar, I. Fernandez, M. Sadrosadati, and O. Mutlu, "DAMOV: A new methodology and benchmark suite for evaluating data movement bottlenecks," *IEEE Access*, vol. 9, pp. 134 457–134 502, 2021.

[2] NVIDIA Corporation, "cuVSLAM: CUDA-accelerated visual odometry and mapping," arXiv:2506.04359, 2025.

[3] W. A. Wulf and S. A. McKee, "Hitting the memory wall: Implications of the obvious," *ACM SIGARCH Computer Architecture News*, vol. 23, no. 1, pp. 20–24, 1995.

[4] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "Processing data where it makes sense: Enabling in-memory computation," *Microprocessors and Microsystems*, vol. 67, pp. 28–41, 2019.

[5] S. Ghose, A. Boroumand, J. S. Kim, J. Gómez-Luna, and O. Mutlu, "Processing-in-memory: A workload-driven perspective," *IBM Journal of Research and Development*, vol. 63, no. 6, pp. 3:1–3:19, 2019.

[6] J. Do, Y.-S. Kee, J. M. Patel, C. Park, K. Park, and D. J. DeWitt, "Query processing on smart SSDs: Opportunities and challenges," in *ACM SIGMOD International Conference on Management of Data*, 2013, pp. 1221–1230.

[7] B. Gu *et al.*, "Biscuit: A framework for near-data processing of big data workloads," in *International Symposium on Computer Architecture (ISCA)*, 2016, pp. 153–165.

[8] J. Gómez-Luna, I. El Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking a new paradigm: Experimental analysis and characterization of a real processing-in-memory system," *IEEE Access*, vol. 10, pp. 52 565–52 608, 2022.

[9] Y.-C. Kwon *et al.*, "A 20nm 6GB function-in-memory DRAM, based on HBM2 with a 1.2TFLOPS programmable computing unit using bank-level parallelism, for machine learning applications," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2021, pp. 350–352.

[10] S. Lee *et al.*, "A 1ynm 1.25V 8Gb, 16Gb/s/pin GDDR6-based accelerator- in-memory supporting 1TFLOPS MAC operation and various activation functions for deep-learning applications," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2022, pp. 1–3.

[11] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[12] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard, "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309–1332, 2016.

[13] H. Durrant-Whyte and T. Bailey, "Simultaneous localization and mapping: Part I," *IEEE Robotics & Automation Magazine*, vol. 13, no. 2, pp. 99–110, 2006.

[14] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 573–580.

[15] Z. Zhang and D. Scaramuzza, "A tutorial on quantitative trajectory evaluation for visual(-inertial) odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 7244–7251.

[16] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.

[17] J. Ahn, S. Hong, S. Yoo, O. Mutlu, and K. Choi, "A scalable processing-in-memory accelerator for parallel graph processing," in *International Symposium on Computer Architecture (ISCA)*, 2015, pp. 105–117.

[18] J. Ahn, S. Yoo, O. Mutlu, and K. Choi, "PIM-enabled instructions: A low-overhead, locality-aware processing-in-memory architecture," in *International Symposium on Computer Architecture (ISCA)*, 2015, pp. 336–348.

[19] O. Villa, M. Stephenson, D. Nellans, and S. W. Keckler, "NVBit: A dynamic binary instrumentation framework for NVIDIA GPUs," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019, pp. 372–383.

[20] J. Cao, R. Sarkar, R. Kumar, J. Arulraj, and H. Kim, "GPU database systems characterization and optimization," Proceedings of the VLDB Endowment, 2023.

[21] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger, "Evaluation techniques for storage hierarchies," *IBM Systems Journal*, vol. 9, no. 2, pp. 78–117, 1970.

[22] K. Hsieh, E. Ebrahimi, G. Kim, N. Chatterjee, M. O'Connor, N. Vijaykumar, O. Mutlu, and S. W. Keckler, "Transparent offloading and mapping (TOM): Enabling programmer-transparent near-data processing in GPU systems," in *International Symposium on Computer Architecture (ISCA)*, 2016, pp. 204–216.

[23] A. Boroumand *et al.*, "Google workloads for consumer devices: Mitigating data movement bottlenecks," in *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2018, pp. 316–331.

[24] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras," *IEEE Transactions on Robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.

[25] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM," *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.

[26] L. Nardi *et al.*, "Introducing SLAMBench, a performance and accuracy benchmarking methodology for SLAM," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2015, pp. 5783–5790.

[27] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The KITTI dataset," *International Journal of Robotics Research*, vol. 32, no. 11, pp. 1231–1237, 2013.

[28] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. W. Achtelik, and R. Siegwart, "The EuRoC micro aerial vehicle datasets," *International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.

[29] D. Schubert, T. Goll, N. Demmel, V. Usenko, J. Stückler, and D. Cremers, "The TUM VI benchmark for evaluating visual-inertial odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 1680–1687.

[30] A. Handa, T. Whelan, J. McDonald, and A. J. Davison, "A benchmark for RGB-D visual odometry, 3D reconstruction and SLAM," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 1524–1531.

[31] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-sim: An extensible simulation framework for validated GPU modeling," in *International Symposium on Computer Architecture (ISCA)*, 2020, pp. 473–486.

[32] V. Kandiah, S. Peverelle, M. Khairy, J. Pan, A. Manjunath, T. G. Rogers, T. M. Aamodt, and N. Hardavellas, "AccelWattch: A power modeling framework for modern GPUs," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021, pp. 738–753.

[33] J. H. Ward, "Hierarchical grouping to optimize an objective function," *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 236–244, 1963.

Appendix A

Glossary

Arithmetic intensity (AI) FLOPs performed per byte fetched from DRAM; the x-axis of the roofline. Low AI = memory-bound.

Attribution Assigning each memory access to the named data structure (allocation) it falls within.

Bandwidth Data volume movable per unit time (GB/s); contrast *latency*.

Bank (DRAM) A subdivision of a DRAM device; near-bank PiM places compute at each bank to harvest internal bandwidth.

Bundle adjustment (BA) Joint nonlinear least-squares refinement of camera poses and 3-D landmarks; cuVSLAM's numerical core.

Cache (L1/L2) Automatic on-chip copies of recently used data; L1 per SM, L2 chip-wide (25 MB on the primary instrument).

Coalescing Hardware merging of a warp's 32 lane addresses into few 32-byte sectors; its failure mode (scatter) wastes up to 8× bandwidth.

Cold-persistent Data with session lifetime touched episodically (the keyframe database).

Compulsory miss A first-touch cache miss no cache size can avoid; streaming data consists almost entirely of these.

CoV Coefficient of variation (standard deviation ÷ mean); run-to-run noise measure (0.14% at locked clocks here).

DRAM speed-of-light (DRAM SoL) Achieved DRAM bandwidth as a percentage of the measured peak; ≥50% is treated as saturation.

Footprint The number of distinct 32-byte sectors a launch touches, i.e. its exact working-set size.

G0–G7 The GPU bottleneck taxonomy (Table 4.2).

Ground truth The externally measured true trajectory against which SLAM output is scored.

Hot-persistent Resident data reused every frame (the BA linear system).

Jaccard overlap $|A \cap B|/|A \cup B|$ between consecutive launches' footprints; measures working-set migration.

Kernel One GPU function launched over a grid of threads.

Keyframe A frame promoted into the long-term map; its descriptors enter the loop-closure database.

Lane One of the 32 SIMT execution slots of a warp.

LFMR Last-to-first miss ratio; on a two-level GPU hierarchy, $1 -$ L2 hit rate. Near $1 =$ caches not helping.

Local memory Per-thread DRAM-backed space used for register spills; coalesced by construction; *not* program data.

Loop closure Recognition of a previously visited place, triggering trajectory-wide drift correction via a database scan.

Memory wall The compounding gap between compute throughput and memory speed; the thesis's founding observation.

MPKI DRAM sector fetches per thousand warp instructions.

Occupancy Resident warps as a fraction of the SM maximum; the GPU's latency-hiding budget.

Persistence The lifetime/touch-pattern axis of a data structure: streaming, hot-persistent, or cold-persistent.

PiM / NDP Processing-in-memory / near-data processing; compute placed at or near DRAM.

Reuse distance Distinct sectors touched between successive accesses to the same sector; its CDF predicts hit rate for any cache size.

Roofline min(peak compute, AI$\times$peak bandwidth) performance bound; placement under the sloped segment = memory-bound.

Sector The 32-byte granule of GPU DRAM/L2 access; four per 128-byte line.

Sectors per request Average sectors fetched per warp memory request; ∼4 coalesced, →32 scattered.

SM Streaming multiprocessor; the GPU's core unit (22 on the primary instrument).

Spill (register) Compiler-generated traffic to local memory when per-thread state exceeds the register budget.

Stall reason Hardware-recorded cause of a warp's failure to issue (memory scoreboard, dependency wait, barrier, . . .).

Streaming Per-frame data consumed once; compulsory-miss dominated.

Substrate A candidate execution home: GPU, CPU, DRAM-PiM, scatter-PiM, near-sensor, in/near-storage.

TaggedAllocator This project's allocation-journaling instrumentation: pointer, size, call stack, and data-structure tag for every GPU allocation.

Warp The fixed 32-thread SIMT execution bundle.

Appendix B

Artifact and Reproducibility Description

All software, configurations, and derived data of this thesis are archived in a single repository (`github.com/mij001/cuvslam-stack`, MIT license). The artifact is organized so that *every table in this document regenerates from committed CSV data using only the Python standard library—no GPU, no datasets, no vendor tools required*; capture (which does require the hardware) is separated from analysis by construction.

B.1 Layout

`configs/` The configuration regime: a small set of human-owned base TOML files plus a deterministic mutation pass that materializes the full experiment matrices (141 accuracy configurations, 192 profiling-coverage variants). One TOML fully describes a run: dataset, camera rig, pipeline mode, feature toggles, and the ground-truth evaluation block.

`profiling/harness/` The capture entry point: one command runs any configuration under nsys, ncu (curated metric set, bounded windows), or NVBit (windowed, kernel-filtered address traces), writing versioned result directories with full provenance (clock locks, driver/tool versions, command lines). A workload-*adapter* interface makes the harness generic: cuVSLAM is one adapter; a command adapter runs arbitrary GPU programs through the identical pipeline.

`profiling/analysis/` The stdlib-only analysis library: screening, classification (the decision tree of §4.4, with thresholds defined once and imported live into documentation), roofline, locality (reuse-distance CDFs with memory-space filtering), attribution (the journal-trace join), campaign synthesis, substrate verdicts, residual ranking, and the DAMOV-parity checks. The GPU-free test suite fabricates known inputs for every

module.

`profiling/calibration/` The known-truth archetype suite (§6.5): eight kernels, one per class, with the blind-classification driver.

`profiling/validation/` The clock-domain intervention scripts (§4.7) and the profiler-neutrality drivers.

`patches/` The instrumentation as reviewable diffs: the TaggedAllocator + NVTX enablement patch against the cuVSLAM source tree, and the NVBit `mem_trace` extensions (launch-window and kernel-name gating; the allocation-event sidecar).

`reports/`, `profiling/reports/` Dated, immutable result directories: the 27-sequence characterization campaign, the locality study and its space-filtered re-derivation (both the superseded and corrected tables are retained, with the correction marked in place), the attribution campaign, the 141-run accuracy matrix, the 192-variant neutrality campaign, the substrate synthesis, the energy and host-I/O measurements, and the DAMOV-validation artifacts (cross-device agreement, hierarchical-clustering agreement).

`dashboard/` The evidence explorer: an interactive application presenting every finding with its computed number, the per-run reasoning chain (time share → metrics vs. thresholds → the decision-tree rule that fired → verdict), and a methodology tab whose formulas and thresholds are stamped from the analysis source at build time so interface and code cannot disagree.

B.2 Environment

Primary instrument: NVIDIA RTX 2000 Ada (`sm_89`), driver 575.64.05, CUDA 12.9.1, Nsight Compute 2025.2.1.3, Nsight Systems 2025.3.2, NVBit 1.8. This exact tool–driver pairing is load-bearing (binary instrumentation constrains the driver generation; the driver generation constrains profiler builds) and is verified by an environment doctor that checks each target machine and prints a one-line remediation for every incompatibility it finds. GPU clocks are locked (1620/7001 MHz) for every measurement; ceilings are re-measured per instrument.

B.3 Reproduction paths

Analysis-only (no GPU): clone; run the test suite; regenerate any table in this thesis from the committed CSVs via the corresponding `analysis` module. **Full capture:** configure a target with the environment doctor; run the regime's campaign drivers (all are resumable and idempotent—re-running skips completed steps); the coverage audit then proves the

captures complete before analysis will consume them. **Instrumented builds:** apply the committed patch to the cuVSLAM source tree and build with the pinned container recipe; the baseline and instrumented binaries are kept side by side, and the QoR check of §5.2 re-verifies output equivalence on any rebuild.

Appendix C

Extended Result Tables

The tables below are excerpts of committed artifacts, reproduced for convenience; the full machine-readable versions live in the repository (Appendix B).

C.1 Cross-sequence attribution consistency (excerpt)

Selected rows of the 49-kernel attribution-consistency table: each kernel's modal top global-space tag, the agreement across every sequence it appears in, and its median memory-space split (shared/spill/global).

Table C.1: Attribution consistency across 27 sequences (excerpt).

kernel	modal global tag	agree	shared	spill	global
`cast_image_kernel`	`images_raw`	100%	0%	0%	100%
`cast_image_kernel_rgb`	`pyramid_levels`	100%	0%	0%	100%
`gaussian_scaling_kernel`	`images_raw`	100%	0%	0%	100%
`conv_grad_x_kernel`	`pyramid_levels`	100%	92%	0%	8%
`conv_grad_y_kernel`	`pyramid_levels`	100%	96%	0%	4%
`gftt_values_kernel`	`feature_tracks`	100%	98%	0%	2%
`non_max_suppression`	`feature_tracks`	100%	90%	0%	10%
`accumulateGFTT_kernel`	`feat. sel. scratch`	100%	89%	0%	11%
`sba::build_full_system_1`	`ba_linear_system`	100%	0%	0%	100%
`sba::calc_jacobians`	`ba_linear_system`	100%	4%	0%	96%
`sba::reduced_system_st.2`	`ba_linear_system`	100%	4%	0%	96%
`st_build_cache`	`keyframe_descriptors`	100%	0%	0%	100%
`st_track_with_cache`	`keyframe_descriptors`	100%	0%	93%	7%
`cub::DeviceMergeSort*`	`feature_tracks`	100%	83–97%	0%	3–17%

Table C.2: Quality-of-results check: instrumented (TaggedAllocator) build vs. baseline build, same configurations (APE, m).

configuration	baseline	instrumented	Δ
EuRoC MH_01 inertial SLAM	0.689	0.689	0.0000
EuRoC V1_01 stereo SLAM	0.037	0.037	0.0000
EuRoC V2_02 stereo odometry	0.177	0.177	0.0000
KITTI 00 stereo odometry	6.841	6.605	−0.236[a]
KITTI 06 stereo SLAM	2.312	2.277	−0.035[a]
TUM long_office RGB-D SLAM	0.018	0.021	+0.003

[a] Within the sequence's own plain-rerun scatter (§5.3); kitti00 is the canonical bimodal loop-closure case.

C.2 Instrumented vs. baseline trajectories (QoR excerpt)

C.3 KITTI per-sequence odometry accuracy

Table C.3: KITTI stereo odometry, per sequence: absolute error and segment-relative drift (100–800 m KITTI protocol).

seq.	APE (m)	avgRTE (%)	avgRE (°)
00	6.84	0.892	0.938
01	11.59	1.899	0.386
02	4.55	0.975	0.707
03	3.94	2.265	0.988
04	2.16	1.967	0.090
05	3.31	0.974	0.723
06	2.88	1.248	0.757
07	1.44	0.982	1.131
08	4.57	1.221	0.959
09	3.53	1.238	0.544
10	2.17	1.010	0.661

At the 500 m segment length the corpus-level translational drift is 0.82%, against the 0.85% published KITTI-protocol figure (§5.1). Sequence 01 (highway, sparse features) dominates the absolute-APE average; sequences 01 and 04 close no loops and serve as the loop-free control in the attribution campaign.

C.4 Measured stage map (NVTX, excerpt)

Table C.4: Kernel-to-stage mapping, measured from cuVSLAM's own NVTX ranges.

NVTX stage	kernels
front-end, per frame	`cast_image_kernel_rgb`, `gaussian_scaling`, `conv_grad_x/y`, `lk_track`
keyframe feature refresh	`gftt_values`, `accumulateGFTT`, `downsample_gftt`, `non_max_suppression`, `select_features`, CUB keypoint sort
visual-odometry solver	`matcher::*`, `cast_depth`, `sba::reduced_system_*`, cuSOLVER helpers
windowed bundle adjustment	`sba::build_full_system_*`, `sba::calc_jacobians`, `sba::evaluate_cost`
keyframe ingest	`st_build_cache`
loop closure & optimization	`st_track_with_cache`